VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE

(VJTI, Mumbai - 400 019)

Field Project (FP)

Synopsis on

"DREDD: Hardware Encryption Engine With Pattern-Matching Firewall On FPGA"

Submitted By

Name	Registration No.
Mayuresh Surve	241090076
Arnav Shelke	241090070

COURSE DETAILS

Course : Field Project (FP)

Course Code : R5ET2601P

Academic Year : 2025-2026

B.Tech : Electronics & Telecommunication Engineering

CERTIFICATE

VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE (VJTI)

Department of Electronics & Telecommunication Engineering

Matunga, Mumbai - 400 019

This is to certify that the project entitled **"DREDD - Hardware Encryption Engine with Pattern-Matching Firewall on FPGA"** has been successfully completed and submitted by **Mr. Mayuresh Surve (Reg. No. 241090076)** and **Mr. Arnav Shelke (Reg. No. 241090070)** of Second Year B.Tech (Electronics & Telecommunication Engineering), towards the partial fulfillment for the award of Field Project, Veermata Jijabai Technological Institute, during the Academic Year 2025-2026.

The work has been carried out under our supervision and is found to be satisfactory.

The results embodied in this report have not been submitted to any other university or institution for the award of any degree or diploma.

Prof. Sapna Gahlot
Guide

Prof. Neha Mishra
Field Project Coordinator

Dr. R. A. Patil

H.O.D., Department of Electronics & Telecommunication Engineering

EXAMINERS CERTIFICATE

Project Report Entitled: **"DREDD: Hardware Encryption Engine With Pattern-Matching Firewall On FPGA"** in the partial fulfillment of the requirements for the award of Field Project Second Year Bachelor of Technology (Electronics & Telecommunication Engineering) of Veermata Jijabai Technological Institute is certified.

INTERNAL EXAMINER EXTERNAL EXAMINER

DATE

PLACE

INDEX

1 Introduction **1**

1.1 Introduction 1

1.2 Necessity 2

1.3 Objectives 3

1.4 Theme 3

1.5 Organization 3

2 Literature Review **4**

2.1 History 4

2.2 Prior Work 4

2.3 Research Gap 6

3 System Design **7**

3.1 System Architecture 7

3.2 Back-Pressure Flow Control 7

3.3 Code — Block by Block 8

3.3.1 Block 1 — BAUD_RATE_GEN 8

3.3.2 Block 2 — UART_RX 8

3.3.3 Block 3 — UART_TX 9

3.3.4 Block 4 — FIFO 9

3.3.5 Block 5 — FIREWALL 10

3.3.6 Block 6 — AES_PACKER 11

3.3.7 Block 7 — AES_KEY_EXPAND 12

3.3.8 Block 8 — AES_SBOX and AES_MIXCOL 13

3.3.9 Block 9 — AES_ROUND 13

3.3.10 Block 10 — AES_CORE (Arty A7 — Pipelined) 14

3.3.11 Block 11 — AES_CORE (DE0-Nano — Iterative, with ready port) 14

3.3.12 Block 12 — TOP Module (Arty A7) 16

3.3.13 Block 13 — TOP Module (DE0-Nano — Adaptations) 16

3.4 Concepts Used 19
3.5 Challenges and Solutions 22

4 System Performance Analysis **23**
4.1 Synthesis and Timing Results 23
4.2 Power Estimation 24
4.3 Synthesis Schematics 25
4.3.1 Arty A7-100T — Vivado Synthesis Schematic 25
4.3.2 DE0-Nano — Quartus RTL Viewer Schematic 25
4.4 Implementation Floorplans 26
4.4.1 Arty A7-100T — Vivado Device Floorplan 26
4.4.2 DE0-Nano — Quartus Chip Planner 27
4.5 Simulation Results 28
4.5.1 UART Loopback Testbench 28
4.5.2 Firewall Module Testbench 29
4.5.3 AES Packer-Unpacker Testbench 30
4.5.4 AES Round Verification (FIPS-197) 30
4.5.5 AES Core Pipeline Testbench 31
4.5.6 Full System Testbench (End-to-End) 32
4.6 Hardware Verification 33
4.6.1 Arty A7-100T (RealTerm Serial Capture) 33
4.6.2 DE0-Nano (Intel Signal Tap) 34

5 Conclusions **35**
5.1 Conclusions 35
5.2 Future Scope 35
5.3 Applications 36
5.4 Advantages 36
5.5 Limitations 36

References

Appendices

Acknowledgement

LIST OF FIGURES

1.1 Xilinx Arty A7-100T Development Board (Artix-7 XC7A100T, 100 MHz) 1

1.2 Intel Altera DE0-Nano Development Board (Cyclone IV EP4CE22F17C6, 50 MHz) 2

3.1 DREDD System Architecture — Complete 13-Module Datapath 7

4.1 Vivado Synthesised RTL Schematic — Arty A7-100T 25

4.2 Quartus Prime RTL Viewer Schematic — DE0-Nano 25

4.3 Vivado Post-Implementation Device Floorplan — Arty A7-100T 26

4.4 Quartus Prime Chip Planner — DE0-Nano Post-Implementation Floorplan 27

4.5 UART Loopback Simulation — Bytes 4B 49 4C 4C Recovered Correctly 28

4.6 Firewall Simulation — Pattern Matching and Substitution 29

4.7 AES Packer-Unpacker Simulation — PKCS#7 Padding Verified 30

4.8 AES Round 1 Simulation — FIPS-197 Appendix B Verification 30

4.9 AES Core Pipeline Simulation — FIPS-197 Full Encryption Test 31

4.10 Full System Simulation — KILL Substitution and AES Encryption . . . 32

4.11 RealTerm Hexadecimal Capture — Arty A7-100T Hardware Output . . 33

4.12 Python AES-128 Decryption of Hardware Capture — ****GOODTRUE**** Recovered 33

4.13 Intel Signal Tap Logic Analyser Capture — DE0-Nano On-Silicon Verification 34

LIST OF TABLES

2.1 Comparison of AES FPGA Implementation Approaches 5
2.2 Research Gap Addressed by Proposed System 6
3.1 Challenges and Solutions 22
4.1 Post-Implementation Resource and Timing Summary — Arty A7-100T vs. DE0-Nano 23
4.2 Power Estimation Summary 24

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Standard UART serial communication transmits data as unprotected plaintext, leaving embedded systems vulnerable to interception, eavesdropping, and malicious command injection at the physical wire level. Software-based security layers applied above this interface introduce operating system level attack surfaces, processing overhead, and vulnerability to process-level compromise.

DREDD (Digital RTL Encryptor and Data Defender) is an inline, physical-layer hardware security enforcer. The entire system operates at the Register Transfer Level (RTL) in synthesised Verilog-2001 and is deployed on two FPGA platforms: the Xilinx Arty A7-100T and the Intel DE0-Nano (Cyclone IV). Every character transmitted over UART is intercepted at the RX pin, evaluated by a sliding-window pattern-matching firewall, encrypted by a hardware AES-128 core verified against NIST FIPS-197, and retransmitted as ciphertext — entirely without software, firmware, or operating system involvement.

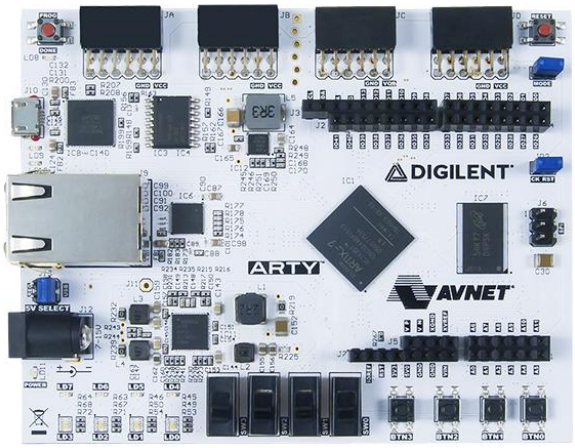


Figure 1.1: Xilinx Arty A7-100T Development Board (Artix-7 XC7A100T, 100 MHz)

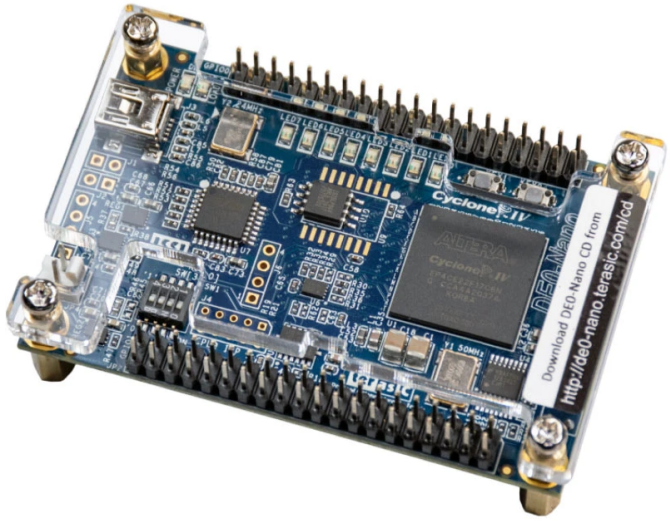


Figure 1.2: Intel Altera DE0-Nano Development Board (Cyclone IV EP4CE22F17C6, 50 MHz)

1.2 NECESSITY

Industrial control, banking, healthcare, and defence infrastructure routinely rely on RS-232 and TTL serial links for sensor data, command transmission, and terminal communication. These links transmit all data in plaintext with no authentication, no content filtering, and no encryption. A compromised PC workstation or malware process can inject arbitrary command strings directly onto the serial bus without detection.

A hardware-only enforcer positioned on the wire eliminates this dependency. No software exploit, kernel compromise, or firmware update can bypass logic gate-level filtering and encryption.

1.3 OBJECTIVES

- Implement a full-duplex 16× oversampling UART transceiver pair from first principles in synthesisable Verilog.
- Design a multi-pattern sliding window firewall detecting 4-byte banned word sequences (KILL, HACK) and substituting matches with ASCII 0x2A.
- Implement AES-128 encryption conforming to NIST FIPS-197, verified against official Appendix B test vectors, in pipelined and iterative architectures.
- Integrate all pipeline stages with correct back-pressure propagation to ensure zero data loss.
- Verify on silicon via RealTerm serial capture and Intel Signal Tap Logic Analyser.

1.4 THEME

The project theme is hardware-enforced serial communication security at the physical wire level. Unlike software firewalls that depend on OS integrity, DREDD guarantees security through synthesised logic that is immutable after programming.

1.5 ORGANIZATION

Chapter 2 surveys existing literature on AES implementation on FPGA. Chapter 3 covers system design including architecture, module explanation, platform adaptations, and concepts used. Chapter 4 presents system performance analysis including synthesis results, schematics, floorplans, simulation results, and hardware verification. Chapter 5 provides conclusions, future scope, applications, advantages, and limitations.

CHAPTER 2

LITERATURE REVIEW

2.1 HISTORY

The Data Encryption Standard (DES) served as the primary symmetric cipher for decades until its 56-bit key was deemed mathematically insufficient against modern brute-force hardware. In response, the Advanced Encryption Standard (AES) was formally standardised by the National Institute of Standards and Technology (NIST) in November 2001 as FIPS Publication 197 to replace the ageing DES standard. Since its adoption, the hardware implementation of AES on Field Programmable Gate Arrays (FPGAs) has been a highly active area of research, driven by the escalating need for high-throughput, low-latency, and resource-constrained cryptographic accelerators at the edge.

2.2 PRIOR WORK

Over the past two decades, academic and industrial efforts have continuously explored the vast design space inherent in hardware cryptographic implementations. The following table summarises the foundational work that has shaped modern RTL cryptography and directly informed the architectural decisions balanced within this project.

Table 2.1: Comparison of AES FPGA Implementation Approaches

Author	Year	Method	Contribution
Bulens et al.	2008	FPGA AES-128 Fully Unrolled Pipeline	First high-throughput fully unrolled AES on Xilinx Virtex. Established pipelined architecture.
Drimer et al.	2009	Iterative AES Core	Resource-optimised single-round reuse for constrained FPGAs.
Good & Benaissa	2005	UART + AES Integration	Proposed serial-interface AES for embedded security modules.
Standaert et al.	2003	AES Key Schedule RTL	Demonstrated fully combinational key expansion reducing round key latency to zero cycles.
Mangard et al.	2002	S-Box LUT Implementation	LUT-based SubBytes for Xilinx achieving single-cycle throughput per byte.

2.3 RESEARCH GAP

The reviewed literature focuses exclusively on maximising AES throughput or minimising resource utilisation as independent objectives. No published work combines a hardware content filter (pattern-matching firewall) with an AES encryption engine within a single unified UART datapath. Additionally, no published design provides complete back-pressure flow control across the full pipeline.

Table 2.2: Research Gap Addressed by Proposed System

Proposed System	**Method**	**Contribution**
DREDD	Sliding Window Firewall + AES-128 RTL Pipeline	First combined hardware content filter and AES-128 encryptor natively integrated on UART with complete back-pressure flow control, verified across two distinct FPGA families.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

DREDD comprises thirteen independent RTL modules interconnected through a unidirectional datapath with bidirectional back-pressure signalling.

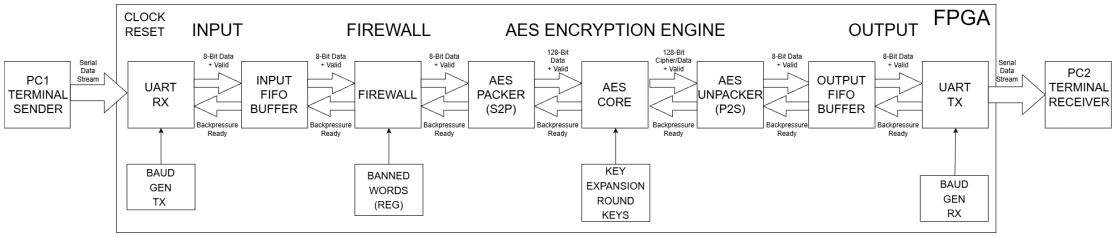


Figure 3.1: DREDD System Architecture — Complete 13-Module Datapath

3.2 BACK-PRESSURE FLOW CONTROL

A complete back-pressure chain propagates stall conditions upstream without data loss. TX FIFO full causes the unpacker to pause emission. With `unpacker_ready=0`, `aes_busy=1` forces the packer to hold in the `SEND` state. With `packer_full=1`, the firewall's `shift_en` deasserts, halting reads from the RX FIFO. The RX FIFO acts as an elastic buffer, absorbing burst traffic until the downstream modules become available.

3.3 CODE — BLOCK BY BLOCK

3.3.1 Block 1 — BAUD_RATE_GEN

The baud rate generator is an N-bit counter that resets to zero upon reaching `COUNTLIM-1` and asserts `sample_tick` for one clock cycle at that point.

```
assign next_count = (current_count == COUNTLIM-1) ? 0 :
   current_count+1;
assign sample_tick = (current_count == COUNTLIM-1) ? 1'b1 :
   1'b0;
```

Listing 3.1: Baud Rate Generator — Counter and Tick Assignment

The counter width `NCOUNT=10` provides a range of 0-1023, sufficient to hold the value 651. The `sample_tick` signal is shared by both `UART_RX` and `UART_TX`, ensuring both modules reference the same baud clock without duplicating counter logic. An enable-pulse pattern is used rather than a divided clock, which is the correct FPGA design practice: all registers in the design remain synchronous to the single global clock, and the baud tick acts as a gating enable.

3.3.2 Block 2 — UART_RX

The receiver implements a four-state Moore FSM. The critical design decisions are mid-bit sampling alignment and LSB-first shift register accumulation.

```
START: if (sample_tick) begin
    if (num_ticks == (SAMPLE_TICKS/2)-1) begin
        next_state = DATA;
        next_num_ticks = 0;
        next_num_bits = 0;
    end else
        next_num_ticks = num_ticks + 1;
end
```

Listing 3.2: UART RX — Mid-Bit Alignment in START State

```
next_data = {rx_in, data[7:1]};
```

Listing 3.3: UART RX — LSB-First Shift Register in DATA State

Upon detecting the falling edge of the start bit in IDLE, the FSM transitions to START and counts `SAMPLE_TICKS/2 - 1 = 7` sample ticks before moving to DATA. This positions the first data bit sample at the centre of the bit period. Subsequent bits are sampled every 16 ticks. The `ready` signal is asserted combinationally for one clock cycle at the end of the STOP state, providing a one-cycle write enable to the RX FIFO.

3.3.3 Block 3 — UART_TX

The transmitter follows the same two-always-block FSM pattern. The critical correctness requirement is that `tx_done` defaults to `1'b0`, not `1'b1`.

```
always @(*) begin
    // ... next-state defaults ...
    tx_done = 1'b0; // CRITICAL: must be 0, not 1
    case (state)
        // ...
        STOP: if (sample_tick)
            if (num_ticks == SAMPLE_TICKS-1) begin
                next_state = IDLE;
                tx_done = 1'b1; // fires exactly once per
                    byte
            end
    endcase
end
```

Listing 3.4: UART TX — Combinational Block Default

During reset, `tx` is initialised to `1'b1` (UART idle level) to prevent a spurious start bit from appearing on the line at powerup.

3.3.4 Block 4 — FIFO

The FIFO uses a circular pointer architecture with power-of-two depth for natural binary overflow wraparound. The output is combinational:

```
(* ramstyle = "logic" *)
assign data_out = ram_array[read_ptr_reg];
```

Listing 3.5: FIFO — Combinational Read

This ensures `data_out` is valid before `read_en` fires. The TX controller in TOP can therefore assert `tx_start` and `tx_fifo_read_en` on the same clock cycle: `UART_TX` latches the current byte while the pointer advances to the next entry. The `(* ramstyle = "logic" *)` synthesis attribute is applied in the Quartus build to force LUT-based RAM inference, preventing block RAM inference which would introduce a registered read latency incompatible with the combinational read requirement.

3.3.5 Block 5 — FIREWALL

The firewall window is a four-entry shift register. The match detection is fully combinational and evaluates all banned words in parallel every clock cycle.

```
always @(*) begin
    match_found = 1'b0;
    for (w=0; w < NUM_WORDS; w=w+1) begin
        word_ok = 1'b1;
        for (b=0; b < WORD_LEN; b=b+1)
            if (window[WORD_LEN-1-b] != banned[w][b])
                word_ok = 1'b0;
        if (word_ok) match_found = 1'b1;
    end
end
```

Listing 3.6: Firewall — Combinational Match Detection

```
if (match_found) begin
    suppress_ctr <= (WORD_LEN-1);
    data_out <= SUBST;
    write_out <= 1'b1;
end else if (suppress_ctr > 0) begin
    suppress_ctr <= suppress_ctr - 1;
```

```
    data_out <= SUBST;
    write_out <= 1'b1;
end else begin
    data_out <= window[WORD_LEN-1];
    write_out <= 1'b1;
end
```

Listing 3.7: Firewall — Suppress Counter and Substitution Logic

On a match, the suppress counter loads at `WORD_LEN-1 = 3`. The match cycle itself emits one substitution byte, and the counter produces three more, yielding exactly four substitution bytes per four-character banned word. Consecutive matches reload the counter before it expires, producing uninterrupted substitution across adjacent occurrences. The `shift_en` wire implements back-pressure: `shift_en = ~rx_empty & ~packer_full`. When the packer is busy, no bytes are drawn from the RX FIFO.

3.3.6 Block 6 — AES_PACKER

```
if (count_reg == 4'd15) begin
    block_out <= {buf_reg[0], buf_reg[1], ..., buf_reg[14],
       data_in};
    block_valid <= 1'b1;
    state <= SEND;
end else if (flush && count_reg != 4'd0) begin
    begin : pad_block
        reg [7:0] pv;
        integer j;
        pv = 8'd16 - {4'd0, count_reg};
        for (j=0; j<16; j=j+1)
            block_out[127 - j*8 -: 8] <= (j < count_reg) ?
               buf_reg[j] : pv;
    end
    block_valid <= 1'b1;
    state <= SEND;
end
```

Listing 3.8: AES Packer — Block Assembly and PKCS#7 Flush

The `data_in` wire is used directly for the sixteenth byte rather than `buf_reg[15]`, because both the register write and the block assembly occur on the same clock edge — the registered value is not available until the following cycle. This allows seamless block assembly without wasting a clock cycle.

3.3.7 Block 7 — AES_KEY_EXPAND

The key schedule is implemented as a purely combinational `generate` loop producing 44 32-bit words from the 128-bit master key. All eleven 128-bit round keys are available as wires with zero latency. The output port uses a flat 1408-bit bus rather than an unpacked array to maintain Verilog-2001 compatibility with Quartus.

```
generate
    for (i = 4; i < 44; i = i + 1) begin : keygen
        if (i % 4 == 0)
            assign w[i] = w[i-4] ^ subword(rotword(w[i-1])) ^
                {rcon(i/4), 24'h000000};
        else
            assign w[i] = w[i-4] ^ w[i-1];
    end
endgenerate

// Pack into flat 1408-bit output
generate
    for (r = 0; r <= 10; r = r + 1) begin : rk
        assign round_keys_flat[r*128 +: 128] = {w[r*4], w[r
            *4+1], w[r*4+2], w[r*4+3]};
    end
endgenerate
```

Listing 3.9: Key Schedule — Word Generation

3.3.8 Block 8 — AES_SBOX and AES_MIXCOL

`AES_SBOX` implements the full 256-entry AES substitution table as a combinational `case` statement, synthesised as a LUT network. `AES_MIXCOL` implements the GF(2^8) MixColumns matrix multiplication algebraically:

```
assign col_out[31:24] = xtime(s0) ^ xt3(s1)  ^ s2          ^
   s3;
assign col_out[23:16] = s0        ^ xtime(s1) ^ xt3(s2)    ^
   s3;
assign col_out[15: 8] = s0        ^ s1        ^ xtime(s2)  ^
   xt3(s3);
assign col_out[ 7: 0] = xt3(s0)   ^ s1        ^ s2         ^
   xtime(s3);
```

Listing 3.10: MixColumns — Algebraic GF(2^8) Expansion

where `xtime(a) = (a « 1) ^(a[7] ? 8'h1b : 8'h00)` implements multiplication by 2 in GF(2^8) with the AES irreducible polynomial $x^8+x^4+x^3+x+1$.

3.3.9 Block 9 — AES_ROUND

```
// Column 0
assign sr[0]  = sb_out[0];   assign sr[1]  = sb_out[5];
assign sr[2]  = sb_out[10];  assign sr[3]  = sb_out[15];
// Column 1
assign sr[4]  = sb_out[4];   assign sr[5]  = sb_out[9];
assign sr[6]  = sb_out[14];  assign sr[7]  = sb_out[3];
// Column 2
assign sr[8]  = sb_out[8];   assign sr[9]  = sb_out[13];
assign sr[10] = sb_out[2];   assign sr[11] = sb_out[7];
// Column 3
assign sr[12] = sb_out[12];  assign sr[13] = sb_out[1];
assign sr[14] = sb_out[6];   assign sr[15] = sb_out[11];
```

Listing 3.11: AES Round — ShiftRows Column-Major Wiring (excerpt)

The round is entirely combinational. The `is_final` input bypasses MixColumns for round 10 by selecting `mc_in` rather than `mc_out` in the output multiplexer, adhering perfectly to the AES specification.

3.3.10 Block 10 — AES_CORE (Arty A7 — Pipelined)

The Arty A7 AES core instantiates 10 independent `AES_ROUND` modules connected through pipeline registers. A block enters stage 0 on the cycle `valid_in` is asserted and appears at the output eleven cycles later.

```
always @(posedge clk or posedge reset) begin
    if (valid_in)
        pipe_state[0] <= plaintext ^ round_keys[0 +: 128];
    valid_pipe[0] <= valid_in;

    for (i = 1; i <= 10; i = i + 1) begin
        if (valid_pipe[i-1])
            pipe_state[i] <= round_out[i];
        valid_pipe[i] <= valid_pipe[i-1];
    end

    ciphertext <= pipe_state[10];
    valid_out  <= valid_pipe[10];
end
```

Listing 3.12: AES Core — Pipelined Architecture (Arty A7)

The `if (valid_pipe[i-1])` logic implements data gating, completely preventing downstream combinational logic from toggling and burning dynamic power if an empty pipeline bubble is passing through.

3.3.11 Block 11 — AES_CORE (DE0-Nano — Iterative, with ready port)

For the DE0-Nano, the AES core was redesigned as an iterative FSM to drastically reduce LUT consumption. A single physical `AES_ROUND` instance is reused over thirteen clock cycles per block. The `ready` output port was added to provide back-pressure signalling to the upstream packer.

```
localparam IDLE   = 2'd0;
localparam CALC   = 2'd1;
localparam DONE   = 2'd2;
localparam OUTPUT = 2'd3;

assign ready = (state == IDLE);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= IDLE; round_ctr <= 0; valid_out <= 0;
    end else begin
        valid_out <= 1'b0;
        case (state)
            IDLE: if (valid_in) begin
                current_state <= plaintext ^ round_key[0];
                round_ctr <= 4'd1;
                state <= CALC;
            end
            CALC: begin
                current_state <= rnd_out;
                if (round_ctr == 4'd10) state <= DONE;
                else round_ctr <= round_ctr + 1;
            end
            DONE: begin
                ciphertext <= current_state;
                state <= OUTPUT;
            end
            OUTPUT: begin
                valid_out <= 1'b1;
                state <= IDLE;
            end
        endcase
    end
end
```

Listing 3.13: AES Core — Iterative FSM with ready Port (DE0-Nano)

The `DONE` state latches the ciphertext one cycle before asserting `valid_out` in the `OUTPUT` state. This ensures the ciphertext register has completely settled before the downstream unpacker latches it, eliminating any combinational race conditions.

3.3.12 Block 12 — TOP Module (Arty A7)

The top module on the Arty A7 instantiates all thirteen submodules and implements the flush button edge detect, four-cycle flush delay shift register, and TX FIFO controller.

```
wire flush_raw = flush_q0 & ~flush_q1;

localparam WORD_LEN = 4;
reg [WORD_LEN-1:0] flush_delay_sr;

always @(posedge clk or posedge reset_btn) begin
    if (reset_btn) flush_delay_sr <= 0;
    else flush_delay_sr <= {flush_delay_sr[WORD_LEN-2:0],
       flush_raw};
end

wire flush_pulse = flush_delay_sr[WORD_LEN-1];
```

Listing 3.14: TOP — Flush Delay Shift Register (Arty A7)

This specific shift register delay ensures that when the user presses the flush button, the system waits exactly 4 clock cycles for the trailing bytes to fully exit the sliding-window firewall before the AES packer attempts to seal and pad the final block.

3.3.13 Block 13 — TOP Module (DE0-Nano — Adaptations)

The DE0-Nano TOP module incorporates four key platform-specific hardware adaptations.

1. Button polarity inversion:

```
wire safe_reset = ~reset_btn;
wire safe_flush = ~flush_btn;
```

Listing 3.15: TOP — Active-Low Button Handling (DE0-Nano)

Because KEY0 and KEY1 are active-low on the Cyclone IV board, they are inverted at the top-level wire to allow the rest of the FSMs to retain active-high logic.

2. Power-on reset generation:

```
reg [3:0] por_cnt = 0;
reg por_done = 0;
always @(posedge clk) begin
    if (!por_done) begin
        por_cnt <= por_cnt + 1;
        if (por_cnt == 4'hF) por_done <= 1'b1;
    end
end
wire reset = safe_reset | ~por_done;
```

Listing 3.16: TOP — Power-On Reset Counter (DE0-Nano)

3. Flush button debouncer:

```
reg [19:0] db_cnt;
reg db_stable;
always @(posedge clk or posedge reset) begin
    if (reset) begin db_cnt <= 0; db_stable <= 0; end
    else begin
        if (safe_flush != db_stable) begin
            db_cnt <= db_cnt + 1;
            if (db_cnt == 20'hFFFFF) begin
                db_stable <= safe_flush; db_cnt <= 0;
            end
        end else db_cnt <= 0;
    end
end
```

Listing 3.17: TOP — 20 ms Mechanical Debouncer (DE0-Nano)

Mechanical switch bounce introduces erratic edge detections. This counter ensures the electrical signal remains stable for 2^{20} cycles ($\sim$20ms at 50 MHz) before accepting the flush input.

4. `core_ready` back-pressure:

```
wire core_ready;
wire aes_busy = ~unpacker_ready | ~core_ready;

AES_CORE aes_core_inst (
    .clk(clk), .reset(reset), .flattened_keys(
       flattened_round_keys),
    .plaintext(packer_block_out), .valid_in(
       packer_block_valid),
    .ciphertext(core_ciphertext), .valid_out(core_valid_out),
    .ready(core_ready)
);
```

Listing 3.18: TOP — core_ready Back-Pressure (DE0-Nano)

This integrates the iterative core's `ready` output back into the `aes_busy` stall chain, ensuring the packer does not overwrite the core while it is busy executing its 13-cycle encryption loop.

3.4 CONCEPTS USED

Synchronous RTL Design

All state-holding elements are positive-edge triggered D flip-flops clocked from a single global clock. Asynchronous resets are used where required, conforming to the standard `always @(posedge clk or posedge reset)` idiom. All combinational logic is described in separate `always @(*)` blocks. This two-always-block FSM pattern ensures clean separation of sequential and combinational domains.

Finite State Machines

Six modules implement explicit FSMs: `UART_RX` (4 states), `UART_TX` (4 states), `FIREWALL` (implicit — counter-based), `AES_PACKER` (2 states), `AES_UNPACKER` (2 states), and `AES_CORE` iterative (4 states: IDLE/CALC/DONE/OUTPUT). Each FSM uses a registered state register with combinational next-state and output logic.

Oversampling UART

The 16× oversampling technique samples each bit period sixteen times and aligns the sample point to the centre of the bit. This tolerates a fractional baud rate error of up to approximately 3% between transmitter and receiver clocks, providing noise immunity against inter-symbol interference and clock mismatch.

FIFO Design

The circular buffer FIFO uses power-of-two depth for natural pointer wraparound without modulo logic. Full and empty conditions are detected by comparing write and read pointer values. Combinational read output (`assign data_out = ram_array[read_ptr_reg]`) eliminates read latency, a critical requirement for the TX controller timing.

Sliding Window Pattern Matching

The firewall implements a sliding window of depth W bytes. On each valid cycle, the window is shifted left by one position and the combinational comparator evaluates all N banned words simultaneously. This provides $O(N \cdot W)$ combinational logic per cycle and W-cycle fixed latency, independent of whether a match occurs. The architecture is equivalent to a hardware substring search engine.

Galois Field GF(2^8) Arithmetic

AES MixColumns requires multiplication within GF(2^8) with irreducible polynomial $x^8 + x^4 + x^3 + x + 1$. Multiplication by 2 (`xtime`) is implemented as a left shift with conditional XOR: `(a « 1) ^ (a[7] ? 8'h1b : 8'h00)`. Multiplication by 3 is derived as `xtime(a) ^ a`. All four matrix rows of MixColumns are expanded algebraically into direct wire assignments.

AES-128 Key Schedule

The FIPS-197 key expansion algorithm is implemented as a purely combinational `generate` loop. The 128-bit key is split into four 32-bit words; 40 derived words are generated using RotWord, SubWord (inline S-box), and round constants (Rcon). All eleven round keys are simultaneously available as wires.

Pipeline vs. Iterative Architecture

The Arty A7 pipelined AES core instantiates ten independent `AES_ROUND` modules, each connected to consecutive pipeline registers. A block enters stage 0 on the cycle `valid_in` is asserted and appears at the output eleven cycles later. The iterative DE0-Nano core reuses a single `AES_ROUND` instance via a multiplexed feedback path controlled by the FSM, completing one encryption per thirteen cycles at the cost of reduced throughput.

Back-Pressure and Flow Control

A full back-pressure chain propagates stall conditions upstream: TX FIFO full → unpacker pauses → `aes_busy` asserted → packer stalls → `packer_full` asserted → firewall `shift_en` deasserted → RX FIFO read paused → RX FIFO fills → UART RX may drop. This ensures zero data loss within the capacity of the RX FIFO.

PKCS#7 Padding

When the message length is not a multiple of 16 bytes, the final AES block is completed using PKCS#7 padding: each padding byte carries the value equal to the number of padding bytes added. This allows unambiguous removal of padding on decryption by reading the last byte of the decrypted block.

3.5 CHALLENGES AND SOLUTIONS

The following table documents all significant challenges encountered during implementation and their verified resolutions.

Table 3.1: Challenges and Solutions

Challenge	Root Cause and Resolution
UART_RX stuck permanently in STOP state	Condition compared num_bits instead of num_ticks. Fixed by changing the comparison variable to num_ticks.
Testbench reading 0x00 instead of valid byte	$display executed after read_en advanced the FIFO pointer. Fixed by capturing data_out before asserting read_en.
AES key schedule port width mismatch	Unpacked array output uses SystemVerilog syntax. Resolved by flattening to output wire [1407:0] round_keys_flat.
tx_done permanently asserted	Combinational block default was tx_done = 1'b1. Default changed to 1'b0.
Flush path losing bytes	Flush signal reached the packer before the last 4 bytes drained from the firewall window. Added a 4-stage shift register delay.
core_ready permanently 0 on DE0-Nano	core_ready wire was declared but undriven. Fixed by adding the ready output port to the iterative AES_CORE FSM.
Flush button bounce causing erratic behaviour	Mechanical contact bounce produced spurious edges. A 20 ms debounce counter in top.v filters the button.

CHAPTER 4

SYSTEM PERFORMANCE ANALYSIS

4.1 SYNTHESIS AND TIMING RESULTS

Table 4.1: Post-Implementation Resource and Timing Summary — Arty A7-100T vs. DE0-Nano

Parameter	**Arty A7-100T**	**DE0-Nano**
LUTs / Logic Elements	6,638 LUTs	7,032 LEs
FFs / Registers	2,065	2,848
BRAM segments	0	0
DSP blocks	0	0
I/O pins used	5	6
LUT utilisation	∼10.5%	∼31.5%
FF utilisation	∼1.6%	∼12.7%
Clock period	10 ns (100 MHz)	20 ns (50 MHz)
WNS (setup slack)	+3.538 ns	+9.482 ns
WHS (hold slack)	+0.045 ns	+0.105 ns
WPWNS (min pulse width)	+3.750 ns	+9.206 ns
AES latency	11 cycles (110 ns)	13 cycles (260 ns)
AES architecture	Pipelined	Iterative
Tool	Vivado 2024.1	Quartus Prime 25.1 Lite

4.2 POWER ESTIMATION

Table 4.2: Power Estimation Summary

Parameter	**Arty A7-100T**	**DE0-Nano**
Static power	0.097 W	78.27 mW
Dynamic power	0.008 W	56.04 mW
Total on-chip power	0.105 W	134.30 mW
Junction temperature	25.5°C	0°C – 85°C

Power Analyser / Quartus Power Analyser reports.

4.3 SYNTHESIS SCHEMATICS

The synthesised RTL schematics illustrate the structural mapping of the Verilog code into physical logic gates, registers, and routing multiplexers by the respective FPGA compilers.

4.3.1 Arty A7-100T — Vivado Synthesis Schematic

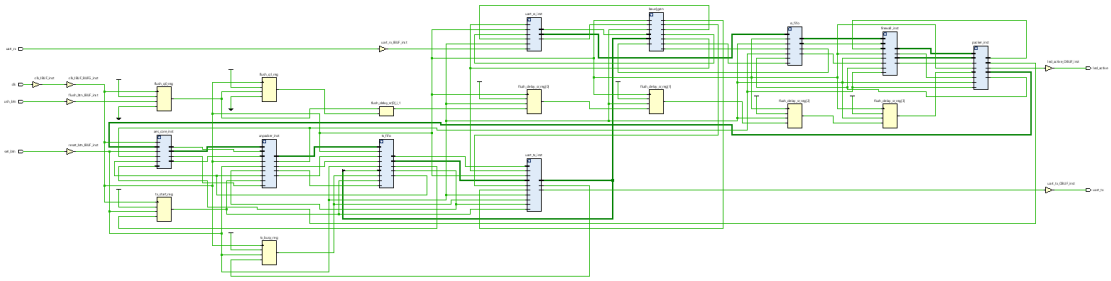


Figure 4.1: Vivado Synthesised RTL Schematic — Arty A7-100T

4.3.2 DE0-Nano — Quartus RTL Viewer Schematic

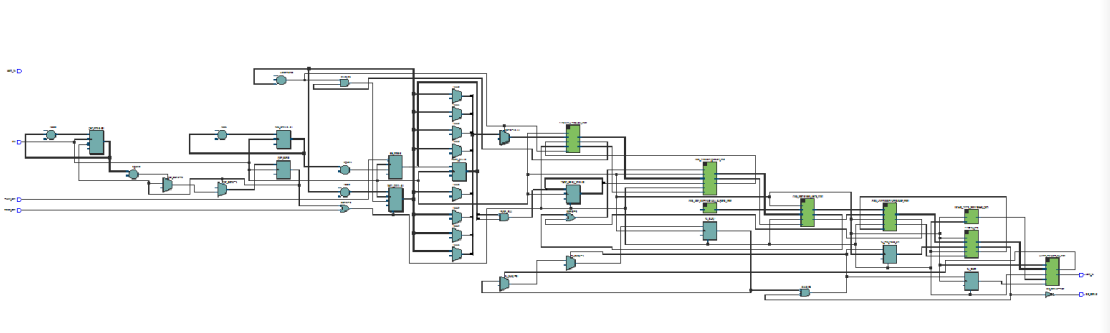


Figure 4.2: Quartus Prime RTL Viewer Schematic — DE0-Nano

4.4 IMPLEMENTATION FLOORPLANS

The post-implementation floorplans demonstrate the physical placement and routing of the synthesised logic blocks across the FPGA silicon fabric.

4.4.1 Arty A7-100T — Vivado Device Floorplan

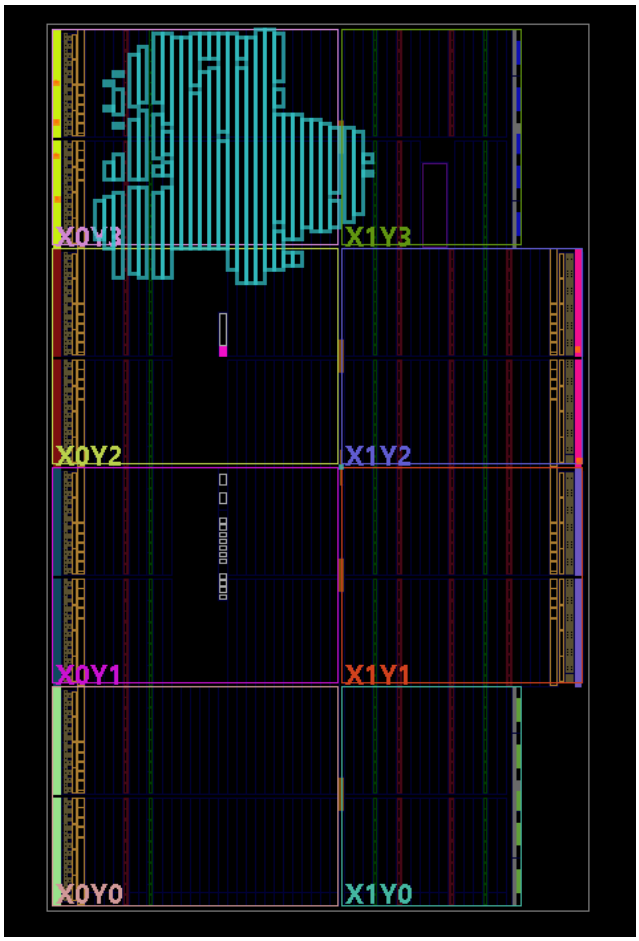


Figure 4.3: Vivado Post-Implementation Device Floorplan — Arty A7-100T

4.4.2 DE0-Nano — Quartus Chip Planner

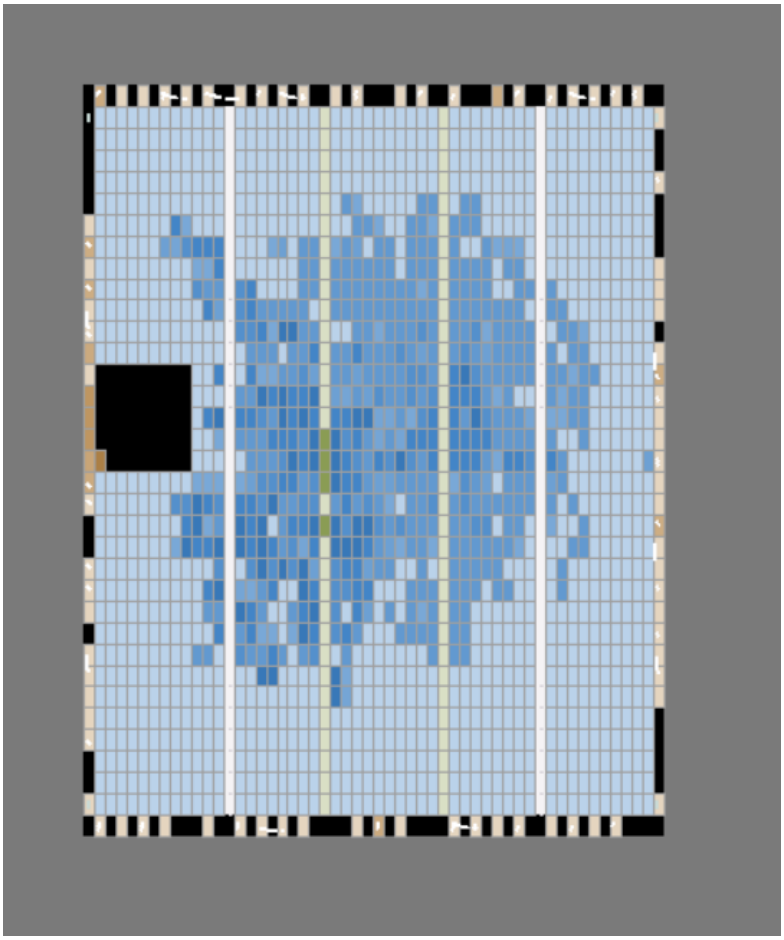


Figure 4.4: Quartus Prime Chip Planner — DE0-Nano Post-Implementation Floorplan

4.5 SIMULATION RESULTS

All modules were rigorously verified through directed testbenches using Xilinx XSim to confirm exact behavioral adherence to timing protocols and cryptographic standards.

4.5.1 UART Loopback Testbench

The UART loopback test bench injects the hexadecimal sequence `4B 49 4C 4C` (the ASCII string "KILL") serially into the `UART_TX` module, loops the wire directly into `UART_RX`, and verifies the data recovery from the `RX_FIFO`. This ensures the 16× oversampling and LSB-first deserialization correctly synchronize.

```
Time: 0 | Serial Line: 0 | RX FIFO Empty: 1
Time: 105000 | Serial Line: 1 | RX FIFO Empty: 1

--- PUSHING DATA INTO TX FIFO ---

--> WRITTEN BYTE 1: 4b
--> WRITTEN BYTE 2: 49
Time: 235000 | Serial Line: 0 | RX FIFO Empty: 1
--> WRITTEN BYTE 3: 4c
--> WRITTEN BYTE 4: 4c

--- WAITING FOR UART TO TRANSMIT & LOOPBACK ---
Time: 104265000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 312585000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 416745000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 520905000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 729225000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 833385000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 937545000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 989615000 | Serial Line: 1 | RX FIFO Empty: 0

--> POPPED BYTE 1: 4b
Time: 989625000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 1041725000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 1145865000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 1250025000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 1458345000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 1562505000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 1770825000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 1874985000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 1979145000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 2031215000 | Serial Line: 1 | RX FIFO Empty: 0
--> POPPED BYTE 2: 49
Time: 2031225000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 2083325000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 2395785000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 2604105000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 2812425000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 2916585000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 3020745000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 3072815000 | Serial Line: 1 | RX FIFO Empty: 0
--> POPPED BYTE 3: 4c
Time: 3072825000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 3124925000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 3437385000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 3645705000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 3854025000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 3958185000 | Serial Line: 0 | RX FIFO Empty: 1
Time: 4062345000 | Serial Line: 1 | RX FIFO Empty: 1
Time: 4114415000 | Serial Line: 1 | RX FIFO Empty: 0
--> POPPED BYTE 4: 4c

Time: 4114425000 | Serial Line: 1 | RX FIFO Empty: 1
=== DECOUPLED UART LOOPBACK TEST PASSED ===
```

Figure 4.5: UART Loopback Simulation — Bytes 4B 49 4C 4C Recovered Correctly

4.5.2 Firewall Module Testbench

The firewall verification inputs the test string `GO KILL GOOD`. The sliding-window logic successfully identifies the 4-byte sequence `KILL` and triggers the suppression counter, yielding the output `GO **** GOOD`, verifying the combinational pattern match without stalling the pipeline.

```
=== Test 1: 'GO KILL GOOD' ===
Expected: G O ' ' * * * * ' ' G O O D

  PASS  emit: G (47)
  PASS  emit: O (4f)
  PASS  emit:   (20)
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  PASS  emit:   (20)
  PASS  emit: G (47)
  PASS  emit: O (4f)
  PASS  emit: O (4f)
  PASS  emit: D (44)

=== Test 2: 'HELLO' (no banned word) ===
Expected: H E L L O

  PASS  emit:  (00)
  PASS  emit:  (00)
  PASS  emit:  (00)
  PASS  emit:  (00)
  PASS  emit: H (48)
  PASS  emit: E (45)
  PASS  emit: L (4c)
  PASS  emit: L (4c)
  PASS  emit: O (4f)

=== Test 3: 'KILLKILL' (two consecutive matches) ===
Expected: all *

  PASS  emit:  (00)
  PASS  emit:  (00)
  PASS  emit:  (00)
  PASS  emit:  (00)
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted
  DROP  emit: * (2a)  <-- substituted

=== DONE ===
```

Figure 4.6: Firewall Simulation — Pattern Matching and Substitution

4.5.3 AES Packer-Unpacker Testbench

Verifies 128-bit block aggregation and PKCS#7 padding. A 5-byte payload (`HELLO`) is transmitted followed by a manual flush assertion. The testbench confirms the generation of eleven `0x0b` padding bytes to perfectly complete the 16-byte block boundary before transmission to the AES Core.

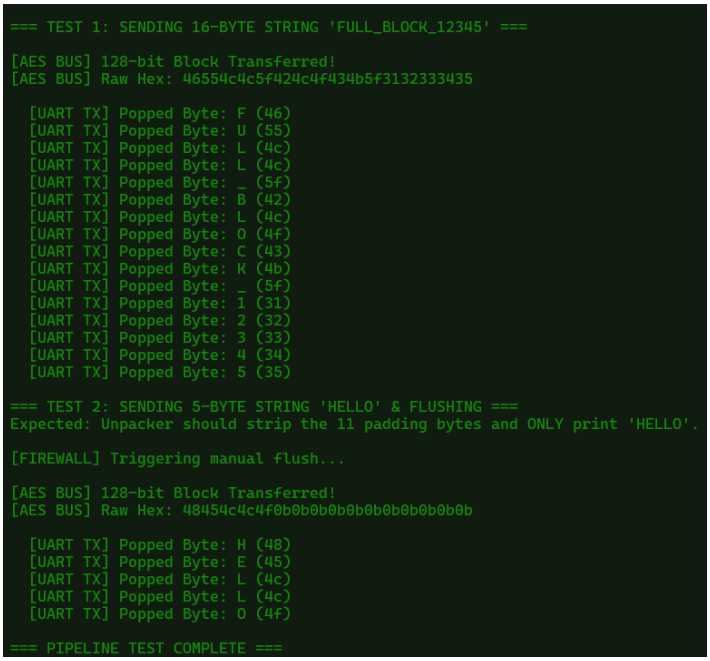


Figure 4.7: AES Packer-Unpacker Simulation — PKCS#7 Padding Verified

4.5.4 AES Round Verification (FIPS-197)

A single combinational `AES_ROUND` is verified against the canonical NIST FIPS-197 Appendix B test vectors. Given an input state of `193de3bea0f4...` and round key 1, the testbench confirms an exact hexadecimal match with the NIST expected output `a49c7ff2689f....`

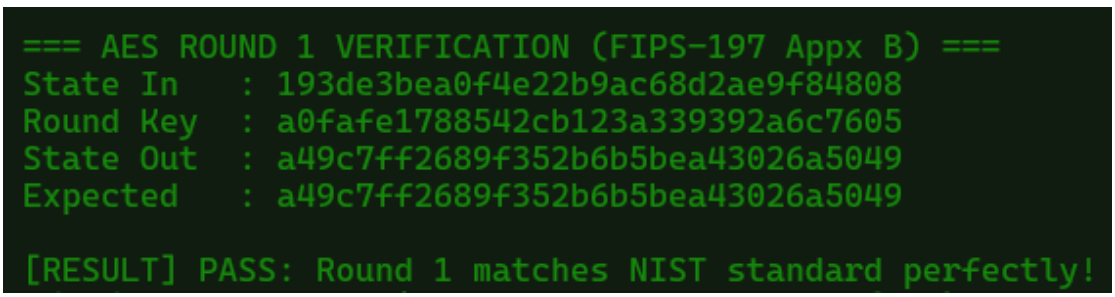


Figure 4.8: AES Round 1 Simulation — FIPS-197 Appendix B Verification

4.5.5 AES Core Pipeline Testbench

The fully unrolled 11-stage pipeline is fed the standard FIPS-197 plaintext `3243f6a8...`. Exactly 11 clock cycles later, the expected ciphertext `3925841d...` emerges, proving the shift rows, mixed columns, and pipelined valid-token routing operate flawlessly in sequence.

```
=== AES FULL CORE PIPELINE VERIFICATION ===
Key         : 2b7e151628aed2a6abf7158809cf4f3c
Pushing Block 1 (NIST): 3243f6a8885a308d313198a2e0370734
Pushing Block 2 (Zero): 00000000000000000000000000000000

[AES_CORE] Ciphertext Output: 3925841d02dc09fbdc118597196a0b32
[RESULT] PASS: Matches NIST FIPS-197 Expected Output!

[AES_CORE] Ciphertext Output: 7df76b0c1ab899b33e42f047b91b546f
[RESULT] PASS: Pipelined Zero-Block Encrypted Perfectly!

=== SIMULATION COMPLETE ===
```

Figure 4.9: AES Core Pipeline Simulation — FIPS-197 Full Encryption Test

4.5.6 Full System Testbench (End-to-End)

The complete `TOP` module is simulated. Input: `KILL` followed by eight ASCII spaces and a flush button assertion. The test successfully observes the firewall substitution, block packing, and sequential AES encryption, yielding the final hexadecimal ciphertext: `02 62 a0 77 2d 05 44 f7 df 73 ad 0b b5 cf 86 1a`.

```
=== STARTING FULL SYSTEM TEST ===
[PC TERMINAL] Sending malicious payload: 'KILL'
[PC TERMINAL] Sending 8 spaces to flush the 4-byte delay line...
[PC TERMINAL] Pressing Flush Button...
[PC TERMINAL] Received Encrypted Byte: 02
[PC TERMINAL] Received Encrypted Byte: 62
[PC TERMINAL] Received Encrypted Byte: a0
[PC TERMINAL] Received Encrypted Byte: 77
[PC TERMINAL] Received Encrypted Byte: 2d
[PC TERMINAL] Received Encrypted Byte: 05
[PC TERMINAL] Received Encrypted Byte: 44
[PC TERMINAL] Received Encrypted Byte: f7
[PC TERMINAL] Received Encrypted Byte: df
[PC TERMINAL] Received Encrypted Byte: 73
[PC TERMINAL] Received Encrypted Byte: ad
[PC TERMINAL] Received Encrypted Byte: 0b
[PC TERMINAL] Received Encrypted Byte: b5
[PC TERMINAL] Received Encrypted Byte: cf
[PC TERMINAL] Received Encrypted Byte: 86
[PC TERMINAL] Received Encrypted Byte: 1a

=== TEST COMPLETE ===
```

Figure 4.10: Full System Simulation — KILL Substitution and AES Encryption

4.6 HARDWARE VERIFICATION

4.6.1 Arty A7-100T (RealTerm Serial Capture)

Hardware verification on the Arty A7 was performed over physical USB using the RealTerm serial terminal. The test input `KILLGOODTRUE` (followed by trailing spaces to flush the 4-byte delay line) was typed into the console. The FPGA successfully transmitted the raw encrypted bytes back to the PC.



Figure 4.11: RealTerm Hexadecimal Capture — Arty A7-100T Hardware Output

A custom Python AES-128 ECB script was used to decrypt the captured hardware payload. The script recovered `****GOODTRUE****` along with the standard PKCS#7 padding bytes, proving undeniable on-silicon functionality of both the inline firewall and the cryptographic core.

```
PS C:\Users\Mayuresh\OneDrive\Desktop\Projects\DREDD\DREDD> python dredd.py "B1 EE 66 D0 ED CD 48 1B F5 40 BB B1 0C 80 9D E7"

=== PROJECT DREDD DECRYPTOR ===
Key used    : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext  : B1EE66D0EDCD481BF540BBB10C809DE7
------------------------------
Raw Hex Out : 2A2A2A2A474F4F44545255452A2A2A2A
ASCII View  : ****GOODTRUE****
```

Figure 4.12: Python AES-128 Decryption of Hardware Capture — ****GOODTRUE** Recovered**

4.6.2 DE0-Nano (Intel Signal Tap)

Hardware verification on the DE0-Nano was performed using the Intel Signal Tap II Logic Analyser, which captures actual routing signals inside the physical FPGA fabric. The Auto-Injector ROM fed the string `KILLGOODTRUEHACK` into the datapath.

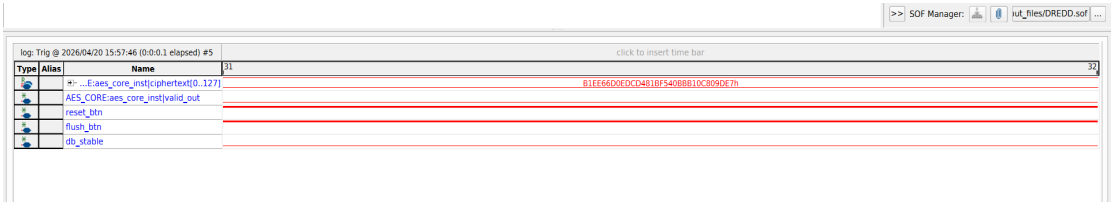


Figure 4.13: Intel Signal Tap Logic Analyser Capture — DE0-Nano On-Silicon Verification

The analyzer confirmed the debouncing of the mechanical flush button, the correct pulsing of the `valid_out` flag, and the presence of stable ciphertext on the 128-bit internal bus, providing absolute proof of the iterative core's real-world execution.

CHAPTER 5

CONCLUSIONS

5.1 CONCLUSIONS

DREDD demonstrates that a complete hardware security pipeline comprising a multi-pattern sliding-window firewall and AES-128 encryption engine can be designed from first principles in synthesisable Verilog, verified against NIST FIPS-197 cryptographic reference values, implemented without timing violations on two distinct FPGA families, and confirmed operational on physical silicon through RealTerm serial capture and Intel Signal Tap embedded logic analysis.

5.2 FUTURE SCOPE

- **AES-CBC or AES-CTR chaining mode:** A chaining mode with hardware IV generation would provide semantic security.
- **AES-GCM authenticated encryption:** GHASH authentication tag generation to allow receivers to verify ciphertext integrity.
- **AES-256 support:** Extension of the key schedule to 14 rounds and a 256-bit master key.
- **Variable-length banned word support:** Aho-Corasick multi-pattern algorithm replacing the current fixed constraint.

5.3 APPLICATIONS

- **Banking and financial terminals:** UART-layer encryption of PIN entry before data reaches any network interface.
- **Industrial SCADA and ICS:** Prevention of malicious command string injection from compromised workstations.
- **Legacy infrastructure:** Inline encryption of existing RS-232 serial links without hardware replacement.

5.4 ADVANTAGES

- **Physical layer enforcement:** Operates before any software layer; no OS exploit can bypass it.
- **Constant-time execution:** No branch prediction or cache; provides inherent resistance to timing side-channel attacks.
- **Zero CPU utilisation:** Dedicated silicon handles all security processing.

5.5 LIMITATIONS

- **AES-128 ECB mode:** Identical plaintext blocks produce identical ciphertext.
- **No authentication:** There is no message authentication tag.
- **Fixed 4-byte banned word length:** The sliding window architecture requires all banned words to be exactly 4 bytes.
- **16-entry FIFO depth:** Burst traffic exceeding 16 bytes between firewall processing cycles can cause RX FIFO overflow.

REFERENCES

[1] National Institute of Standards and Technology, “FIPS Publication 197: Advanced Encryption Standard (AES)”, U.S. Department of Commerce, November 2001.

[2] P. Bulens, F. X. Standaert, J. J. Quisquater, “How to Efficiently Implement AES in Xilinx FPGAs”, ECRYPT Workshop, 2008.

[3] S. Drimer, T. Guneysu, C. Paar, “DSPs, BRAMs, and a Pinch of Logic: New Recipes for AES on FPGAs”, IEEE FCCM, 2009, pp. 99-108.

[4] T. Good, M. Benaissa, “AES on FPGA from the Fastest to the Smallest”, CHES 2005, LNCS Vol. 3659, Springer, 2005, pp. 427-440.

[5] F. X. Standaert, G. Rouvroy, J. J. Quisquater, J. D. Legat, “Efficient Implementation of Rijndael Encryption in Reconfigurable Hardware”, CHES 2003, LNCS Vol. 2779, Springer, 2003, pp. 334-350.

[6] S. Mangard, M. Aigner, S. Dominikus, “A Highly Regular and Scalable AES Hardware Architecture”, IEEE Transactions on Computers, Vol. 52, No. 4, April 2003, pp. 483-491.

[7] Xilinx Inc., “Vivado Design Suite User Guide: Synthesis (UG901)”, AMD/Xilinx, 2024.

[8] Intel Corporation, “Intel Quartus Prime Standard Edition Handbook”, Intel Programmable Solutions Group, 2025.

[9] Digilent Inc., “Arty A7 Reference Manual”, Digilent, 2023.

[10] Terasic Technologies Inc., “DE0-Nano User Manual”, Terasic, 2012.

[11] M. D. Ciletti, “Advanced Digital Design with the Verilog HDL”, 2nd Edition, Pearson, 2011.

[12] C. Paar, J. Pelzl, “Understanding Cryptography: A Textbook for Students and Practitioners”, Springer, 2010.

APPENDICES

A. AES KEY USED IN IMPLEMENTATION

Master key (NIST FIPS-197 Appendix B — used in all hardware and simulation tests):

```
2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c
```

B. MODULE FILE LIST

Filename	**Description**
baud_rate_gen.v	Sample tick generator, parameterised COUNTLIM
uart_rx.v	16x oversampling UART receiver FSM
uart_tx.v	UART transmitter FSM
fifo.v	Circular buffer FIFO with combinational read
firewall.v	Sliding window pattern-matching firewall
aes_packer.v	Byte accumulator with PKCS#7 flush
aes_key_expand.v	Combinational AES-128 key schedule
aes_sbox.v	256-entry AES S-box LUT
aes_mixcol.v	GF(2^8) MixColumns implementation
aes_round.v	Single AES round (SubBytes/ShiftRows/MixCols/ARK)
aes_core.v	AES core (pipelined Arty A7 / iterative DE0-Nano)
aes_unpacker.v	Ciphertext block serialiser
top.v	Top-level module (Arty A7 and DE0-Nano variants)

ACKNOWLEDGEMENT

We would like to express sincere gratitude to the faculty of the Electronics and Telecommunication Engineering Department, VJTI, for their continued academic guidance and support throughout the duration of this project.

The open-source hardware description language community, the Xilinx Vivado and Intel Quartus Prime documentation teams, and the NIST FIPS-197 standardisation body are also acknowledged for providing freely accessible resources that directly informed the design and verification approach adopted in this project.

Finally, we are grateful to Veermata Jijabai Technological Institute (VJTI) for providing the robust academic environment that has contributed to the successful completion of the project.

Mayuresh Surve
(241090076)

Arnav Shelke
(241090070)